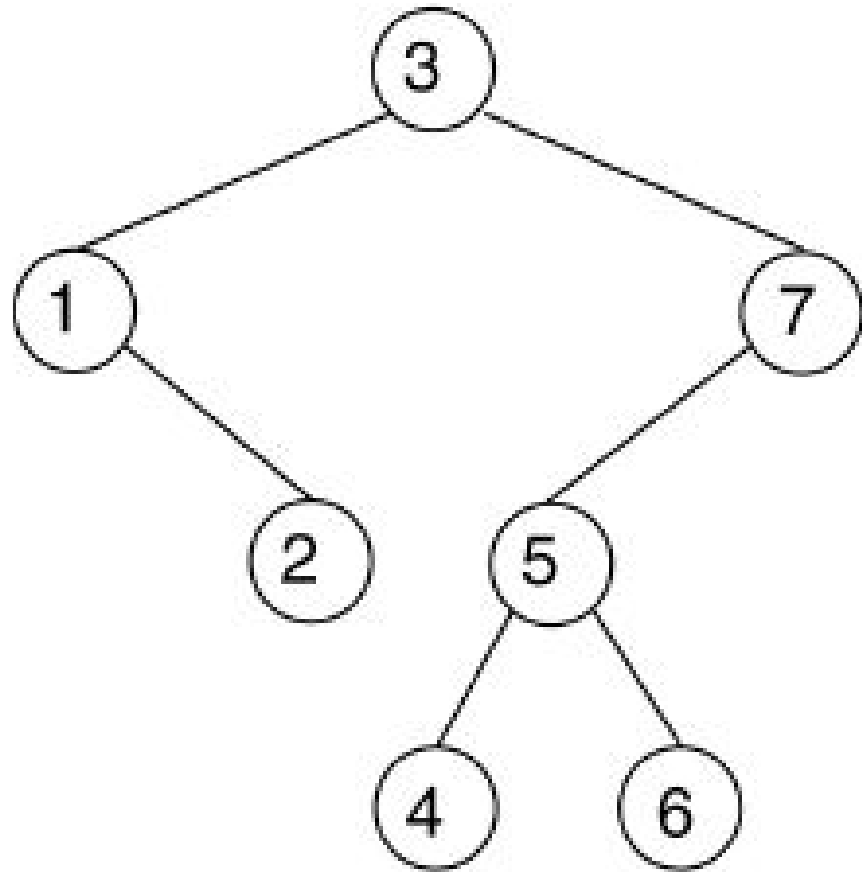


Árvore Binária de Busca

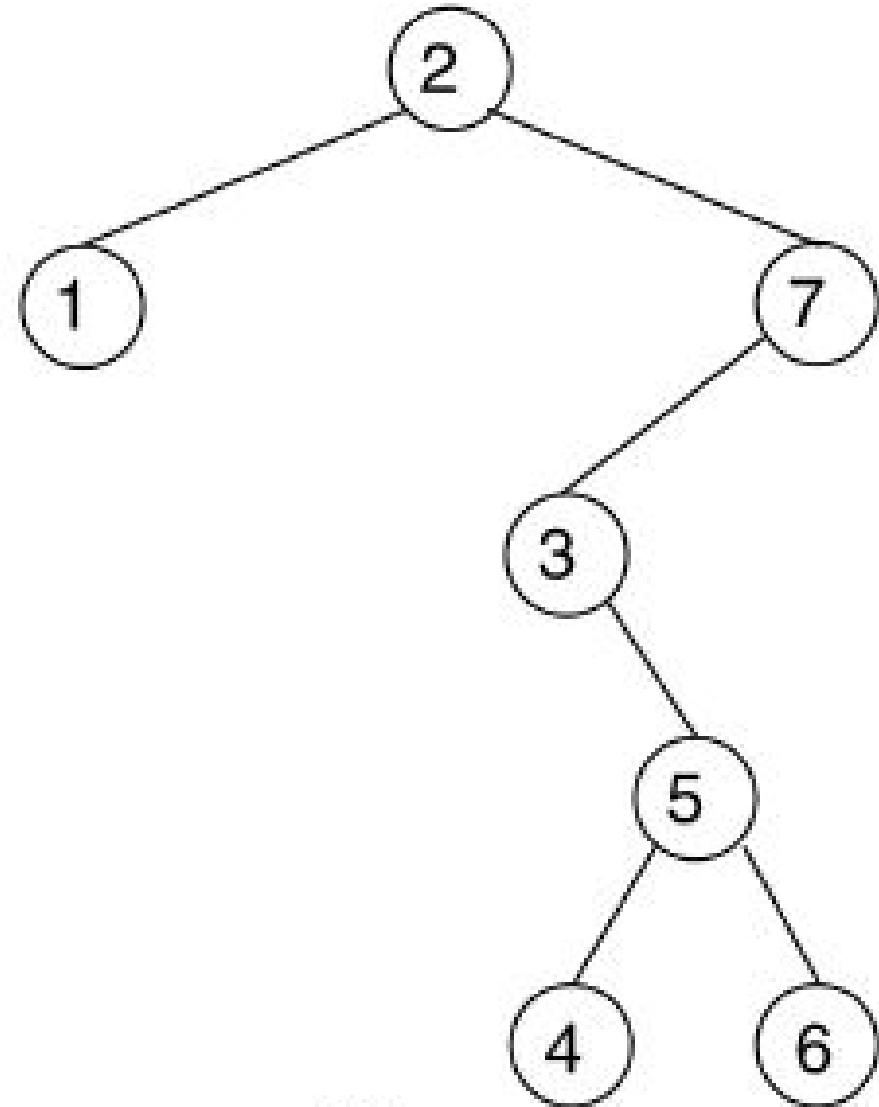
Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

- Seja $S = \{s_1, \dots, s_n\}$ o conjunto de chaves satisfazendo $s_1 < \dots < s_n$. Seja x um valor dado. O objetivo é verificar se $x \in S$ ou não. Em caso positivo, localizar x em S , isto é, determinar o índice j tal que $x = s_j$.
- Empregamos uma árvore rotulada T , com as seguintes características:
 - i. T possui n nós. Cada nó v corresponde a uma chave distinta $s_j \in S$ e possui como rótulo o valor $rt(v) = s_j$ (o rótulo é o valor da chave do nó).
 - ii. Seja um nó v de T . Seja também v_1 , pertencente à subárvore esquerda de v . Então $rt(v_1) < rt(v)$. Se v_2 pertence à subárvore direita de v , $rt(v_2) > rt(v)$.



(a)

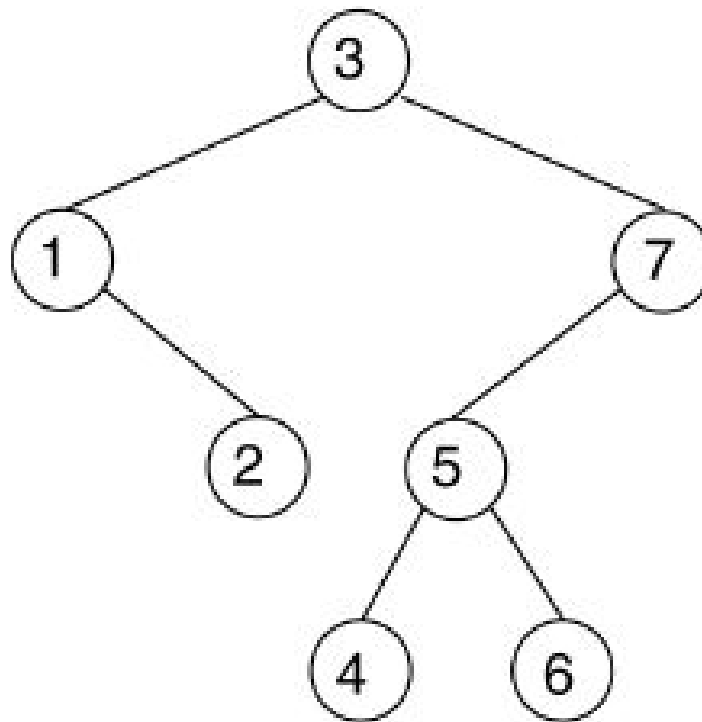


(b)

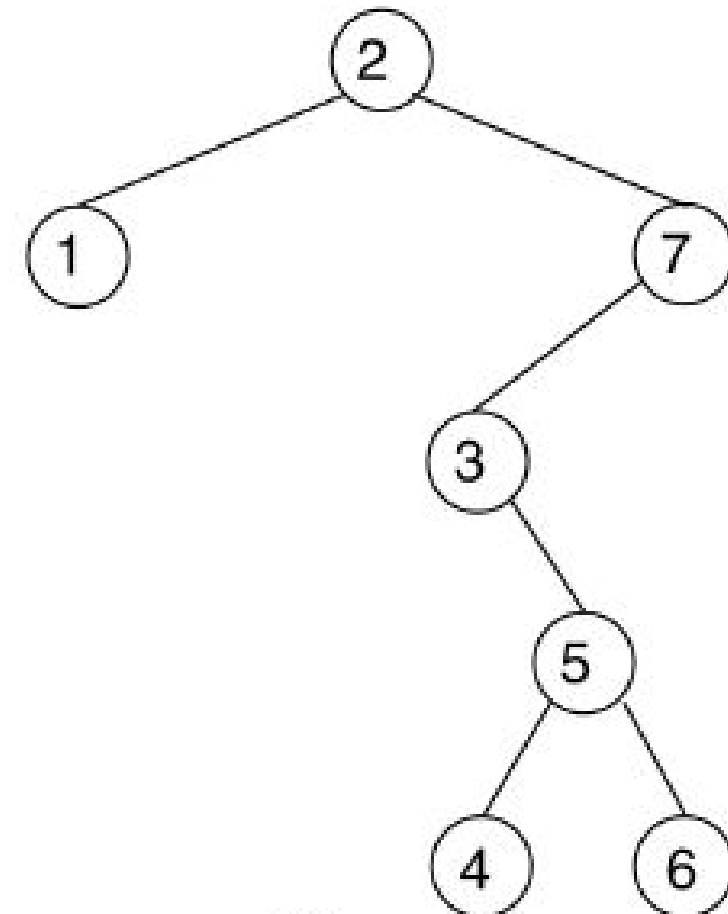
Busca em árvore de binária.
Inicia pela raiz.

Se $x = \text{rt}(v)$ a busca termina.
Caso contrário, se $x < \text{rt}(v)$,
continua com o filho esquerdo,
se $x > \text{rt}(v)$, continua com o
filho direito.

Caso o nó a continuar a busca
não existe, a busca termina e a
árvore não contém a chave
procurada.



(a)



(b)

```
procedimento busca-arvore(x, pt)
    se pt =  $\lambda$  então f := 0
    senão se x == pt.chave então f = 1
        senão se x < pt.chave então
            se pt.esq ==  $\lambda$  então f = 2
            senão
                pt = pt.esq
                pt, f = busca-arvore(x, pt)
        senão se pt.dir ==  $\lambda$  então f = 3
        senão
            pt = pt.dir
            pt, f = busca-arvore(x, pt)

    return pt, f

pt = ptraiiz;
pt, f = busca-arvore(x, pt)
```

procedimento busca-arvore(x, pt)

se pt = λ **então** f := 0

senão se x == pt.chave **então** f = 1

senão se x < pt.chave **então**

se pt.esq == λ **então** f = 2

senão

pt = pt.esq

pt, f = busca-arvore(x, pt)

senão se pt.dir == λ **então** f = 3

senão

pt = pt.dir

pt, f = busca-arvore(x, pt)

return pt, f

pt = ptraiiz;

pt, f = busca-arvore(x, pt)

Complexidade $O(n)$

procedimento busca-arvore(x, pt)

se pt = λ **então** f := 0

senão se x == pt.chave **então** f = 1

senão se x < pt.chave **então**

se pt.esq == λ **então** f = 2

senão

pt = pt.esq

pt, f = busca-arvore(x, pt)

senão se pt.dir == λ **então** f = 3

senão

pt = pt.dir

pt, f = busca-arvore(x, pt)

return pt, f

pt = ptraiiz;

pt, f = busca-arvore(x, pt)

Complexidade $O(n)$

Caso a árvore seja completa, a complexidade será de $O(\log n)$

Inserção em árvore binária de busca

Function inserir(ptraiz, x)

pt = ptraiz; pt, f = busca-arvore(x, pt, f)

se f == 1 **então**

“inserção inválida “

senão new(pt1)

pt1.chave = x; pt1.info = novo-valor

pt1.esq = λ ; pt1.dir = λ

se f == 0 **então** ptraiz = pt1

senão se f == 2 **então**

pt.esq = pt1

senão pt.dir = pt1

return ptraiz


```
typedef struct arvno ArvNo;  
struct arvno{  
    int chave;  
    char info[15];  
    ArvNo* esq;  
    ArvNo* dir;  
};
```

```
ArvNo* ptr Luiz;
```

Em Python:

class Elemento:

```
def __init__(self):  
    self.valor = 0  
    self.esq = None  
    self.dir = None
```

```
def busca(chave, raiz):  
    if (chave < raiz.valor):  
        if ((raiz.esq != None) and (raiz.esq.valor !=  
chave)):  
            pont = busca(chave, raiz.esq)  
        else:  
            pont = raiz  
    else:  
        if ((raiz.dir != None) and (raiz.dir.valor != chave)):  
            pont = busca(chave, raiz.dir)  
        else:  
            pont = raiz  
    return pont
```

```
def incluir(chave, raiz):  
    if (raiz == None):  
        raiz = Elemento()  
        raiz.valor = chave  
        raiz.esq = None  
        raiz.dir = None  
    else:  
        if (raiz.valor == chave):  
            print("Elemento existe na  
arvore")  
        else:
```

```
pont = busca(chave, raiz)  
if (pont.valor != chave):  
    novo = Elemento()  
    novo.valor = chave  
    novo.esq = None  
    novo.dir = None  
    if (chave < pont.valor):  
        pont.esq = novo  
    else:  
        pont.dir = novo  
else:  
    print("Elemento existe na arvore")
```